



SECTION A: NUMPY

Q.1 Create a 1D NumPy array of 20 random integers between 1 and 100. Find the minimum, maximum, mean, and standard deviation without using Python built-in functions — use only NumPy aggregation functions.

```
In [1]: import numpy as np
arr = np.random.randint(1, 101, size=20)
print(arr)
```

```
[34 93 59 12 68 37 18 17 91 66 50 17 77  6 63 82 20  2 85 71]
```

```
In [2]: np.min(arr)
```

```
Out[2]: np.int32(2)
```

```
In [3]: np.max(arr)
```

```
Out[3]: np.int32(93)
```

```
In [4]: np.mean(arr)
```

```
Out[4]: np.float64(48.4)
```

```
In [5]: np.std(arr)
```

```
Out[5]: np.float64(29.948956576148024)
```

Q.2 Create a 4×4 matrix using NumPy. Perform the following operations:

- Extract the diagonal elements
- Replace all even numbers with 0
- Find the index of the maximum value using `argmax()`

```
In [6]: Matrix=np.array([
        [1,2,3,4],
        [4,5,6,7],
        [8,4,5,6],
        [9,1,4,5]
    ])
print(Matrix)
```

```
[[1 2 3 4]
 [4 5 6 7]
 [8 4 5 6]
 [9 1 4 5]]
```

```
In [7]: digonal=np.diagonal(Matrix)
print(digonal)
```

```
[1 5 5 5]
```

```
In [8]: Matrix1=np.where(Matrix%2==0,0,Matrix)
```

```
print(Matrix1)
```

```
[[1 0 3 0]
 [0 5 0 7]
 [0 0 5 0]
 [9 1 0 5]]
```

```
In [9]: np.argmax(Matrix)
```

```
Out[9]: np.int64(12)
```

Q.3 Write a program to create two 3×3 NumPy arrays and perform element-wise addition, subtraction, and matrix multiplication. Display results with proper labels..

```
In [10]: arr1=np.array([
          [1,2,3],
          [4,5,6],
          [7,8,9]
        ])
print(arr1)
arr2=np.array([
          [9,8,7],
          [6,5,4],
          [3,2,1]
        ])
print(arr2)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

```
In [11]: addition=arr1+arr2
print("Addition:",addition)
```

```
Addition: [[10 10 10]
 [10 10 10]
 [10 10 10]]
```

```
In [12]: subtraction=arr1-arr2
print("Subtraction:",subtraction)
```

```
Subtraction: [[-8 -6 -4]
 [-2  0  2]
 [ 4  6  8]]
```

```
In [13]: multiplication=np.matmul(arr1,arr2)
print("Multiplication:",multiplication)
```

```
Multiplication: [[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Q.4 Using np.linspace() and np.zeros() and np.ones(), create three different arrays and stack them vertically using np.vstack(). Display the final stacked array.

```
In [14]: arr_linespace=np.linspace(1,10,5)
         print(arr_linespace)

         arr_zeros=np.zeros(5)
         print(arr_zeros)

         arr_ones=np.ones(5)
         print(arr_ones)
```

```
[ 1.    3.25  5.5   7.75 10. ]
[0.  0.  0.  0.  0.]
[1.  1.  1.  1.  1.]
```

```
In [15]: stacked_arr =np.vstack((arr_linespace,arr_ones,arr_zeros))
         print(stacked_arr)
```

```
[[ 1.    3.25  5.5   7.75 10. ]
 [ 1.    1.    1.    1.    1. ]
 [ 0.    0.    0.    0.    0. ]]
```

SECTION B:PANDAS

Q.5 Create a DataFrame of at least 10 students with columns: Name, Age, Marks, City. Perform the following: • Display the first 5 rows using head() • Show basic statistics using describe() • Filter students who scored more than 75 marks • Sort by Marks in descending order

```
In [16]: import pandas as pd
         data={
             "Name":["A","B","C","D","E","F","G","H","I","J"],
             "Age": [10,12,15,9,7,16,18,19,20,25],
             "Marks": [85, 72, 91, 68, 77, 95, 74, 88, 81, 70],
             "City": ["Delhi", "Mumbai", "Jaipur", "Pune", "Chennai",
                     "Kolkata", "Delhi", "Jaipur", "Mumbai", "Pune"]
         }

         df=pd.DataFrame(data)

         print(df)
```

	Name	Age	Marks	City
0	A	10	85	Delhi
1	B	12	72	Mumbai
2	C	15	91	Jaipur
3	D	9	68	Pune
4	E	7	77	Chennai
5	F	16	95	Kolkata
6	G	18	74	Delhi
7	H	19	88	Jaipur
8	I	20	81	Mumbai
9	J	25	70	Pune

In [17]: `df.head(5)`

Out[17]:

	Name	Age	Marks	City
0	A	10	85	Delhi
1	B	12	72	Mumbai
2	C	15	91	Jaipur
3	D	9	68	Pune
4	E	7	77	Chennai

In [18]: `df.describe()`

Out[18]:

	Age	Marks
count	10.000000	10.000000
mean	15.100000	80.100000
std	5.626327	9.362455
min	7.000000	68.000000
25%	10.500000	72.500000
50%	15.500000	79.000000
75%	18.750000	87.250000
max	25.000000	95.000000

In [19]: `df[df["Marks"]>75]`

Out[19]:

	Name	Age	Marks	City
0	A	10	85	Delhi
2	C	15	91	Jaipur
4	E	7	77	Chennai
5	F	16	95	Kolkata
7	H	19	88	Jaipur
8	I	20	81	Mumbai

In [20]: `df.sort_values(by="Marks", ascending=False)`

Out[20]:

	Name	Age	Marks	City
5	F	16	95	Kolkata
2	C	15	91	Jaipur
7	H	19	88	Jaipur
0	A	10	85	Delhi
8	I	20	81	Mumbai
4	E	7	77	Chennai
6	G	18	74	Delhi
1	B	12	72	Mumbai
9	J	25	70	Pune
3	D	9	68	Pune

Q.6 Create a DataFrame with some missing values intentionally placed in the Marks and City columns. Then:

- Identify missing values using `isnull()`
- Fill missing marks with the column mean using `fillna()`
- Drop rows where City is missing using `dropna()`

In [21]:

```
import numpy as np
data = {
    "Name": ["Aman", "Riya", "Sakshi", "Rahul", "Priya",
             "Karan", "Neha", "Arjun", "Pooja", "Vikas"],
    "Age": [18, 19, 18, 20, 19, 18, 20, 19, 18, 21],
    "Marks": [85, np.nan, 91, 68, np.nan, 95, 74, 88, 81, 70],
    "City": ["Delhi", "Mumbai", np.nan, "Pune", "Chennai",
             "Kolkata", "Delhi", np.nan, "Mumbai", "Pune"]
}

df=pd.DataFrame(data)
print(data)
```

```

df.isnull()
print("Missing values:")
print("\n",df.isnull())

mean_marks = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(mean_marks)
print("\n",df)

df_clean=df.dropna(subset="City")
print("\n",df_clean)

```

```

{'Name': ['Aman', 'Riya', 'Sakshi', 'Rahul', 'Priya', 'Karan', 'Neha', 'Arjun',
'Pooja', 'Vikas'], 'Age': [18, 19, 18, 20, 19, 18, 20, 19, 18, 21], 'Marks': [85, nan, 91, 68, nan, 95, 74, 88, 81, 70], 'City': ['Delhi', 'Mumbai', nan, 'Pune', 'Chennai', 'Kolkata', 'Delhi', nan, 'Mumbai', 'Pune']}
Missing values:

```

	Name	Age	Marks	City
0	False	False	False	False
1	False	False	True	False
2	False	False	False	True
3	False	False	False	False
4	False	False	True	False
5	False	False	False	False
6	False	False	False	False
7	False	False	False	True
8	False	False	False	False
9	False	False	False	False

	Name	Age	Marks	City
0	Aman	18	85.0	Delhi
1	Riya	19	81.5	Mumbai
2	Sakshi	18	91.0	NaN
3	Rahul	20	68.0	Pune
4	Priya	19	81.5	Chennai
5	Karan	18	95.0	Kolkata
6	Neha	20	74.0	Delhi
7	Arjun	19	88.0	NaN
8	Pooja	18	81.0	Mumbai
9	Vikas	21	70.0	Pune

	Name	Age	Marks	City
0	Aman	18	85.0	Delhi
1	Riya	19	81.5	Mumbai
3	Rahul	20	68.0	Pune
4	Priya	19	81.5	Chennai
5	Karan	18	95.0	Kolkata
6	Neha	20	74.0	Delhi
8	Pooja	18	81.0	Mumbai
9	Vikas	21	70.0	Pune

Q.7 Read a CSV file (you may create it manually) containing product sales data with columns: Product, Category, Sales, Region. Perform:

- Group by Category and calculate total Sales using groupby() and agg()
- Find the top 3 products by sales using sort_values() and head().

```
In [22]: df=pd.read_csv("sales_data.csv")
print(df)
```

	Product	Category	Sales	Region
0	Laptop	Electronics	50000	North
1	Mobile	Electronics	35000	South
2	TV	Electronics	40000	East
3	Shirt	Clothing	12000	North
4	Jeans	Clothing	18000	West
5	Shoes	Clothing	15000	South
6	Rice	Grocery	10000	East
7	Oil	Grocery	8000	West
8	Sugar	Grocery	7000	North
9	Biscuits	Grocery	6000	South

```
In [23]: category_sales = df.groupby("Category").agg(
        Total_Sales=("Sales", "sum")
    )
print("\nCategory Sales:",category_sales)
```

Category Sales:	Total_Sales
Category	
Clothing	45000
Electronics	125000
Grocery	31000

```
In [24]: top_3_products = df.sort_values(by="Sales", ascending=False).head(3)

print("\nTop 3 Products by Sales:")
print(top_3_products)
```

Top 3 Products by Sales:

	Product	Category	Sales	Region
0	Laptop	Electronics	50000	North
2	TV	Electronics	40000	East
1	Mobile	Electronics	35000	South

Q.8 Merge two DataFrames: • DataFrame 1: StudentID, Name, Department •
 DataFrame 2: StudentID, Marks, Grade • Merge on StudentID and display students
 who scored Grade 'A'

```
In [25]: df1 = pd.DataFrame({
        "StudentID": [101, 102, 103, 104, 105],
        "Name": ["Mannu", "Riya", "Sakshi", "Rahul", "Priya"],
        "Department": ["CSE", "ECE", "ME", "CSE", "ECE"]
    })

df2 = pd.DataFrame({
        "StudentID": [101, 102, 103, 104, 105],
        "Marks": [85, 72, 91, 68, 88],
```

```

    "Grade": ["A", "B", "A", "C", "A"]
})

merged_df=pd.merge(df1,df2,on="StudentID")
print(merged_df)

grade_a_students = merged_df[merged_df["Grade"] == "A"]
print(grade_a_students)

```

	StudentID	Name	Department	Marks	Grade
0	101	Mannu	CSE	85	A
1	102	Riya	ECE	72	B
2	103	Sakshi	ME	91	A
3	104	Rahul	CSE	68	C
4	105	Priya	ECE	88	A

	StudentID	Name	Department	Marks	Grade
0	101	Mannu	CSE	85	A
2	103	Sakshi	ME	91	A
4	105	Priya	ECE	88	A

SECTION C: DATAVISUALIZATION

Q.9 Using Matplotlib, create the following plots from a student marks dataset (create the dataset yourself):

- A line plot showing marks trend across 5 subjects
- A bar chart comparing marks of 5 students
- Add proper title, xlabel, ylabel, legend, and grid to both plots

```

In [26]: import matplotlib.pyplot as plt

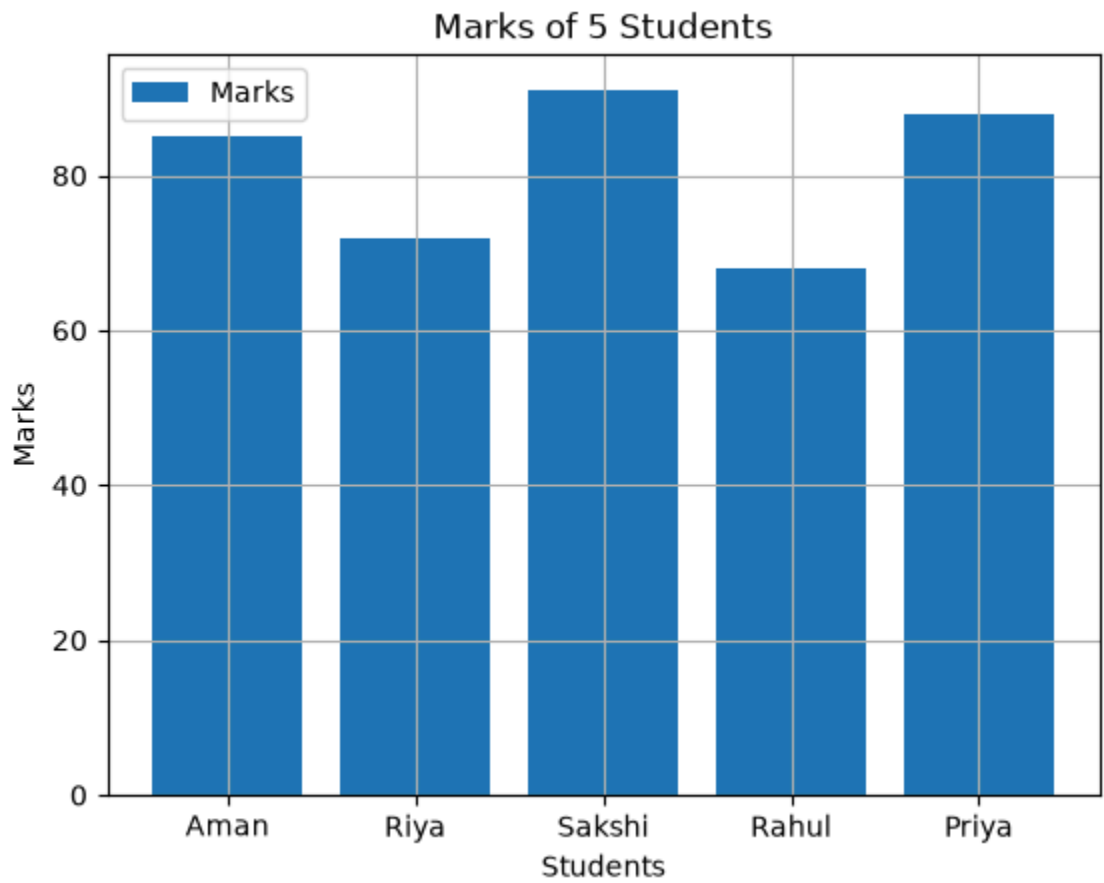
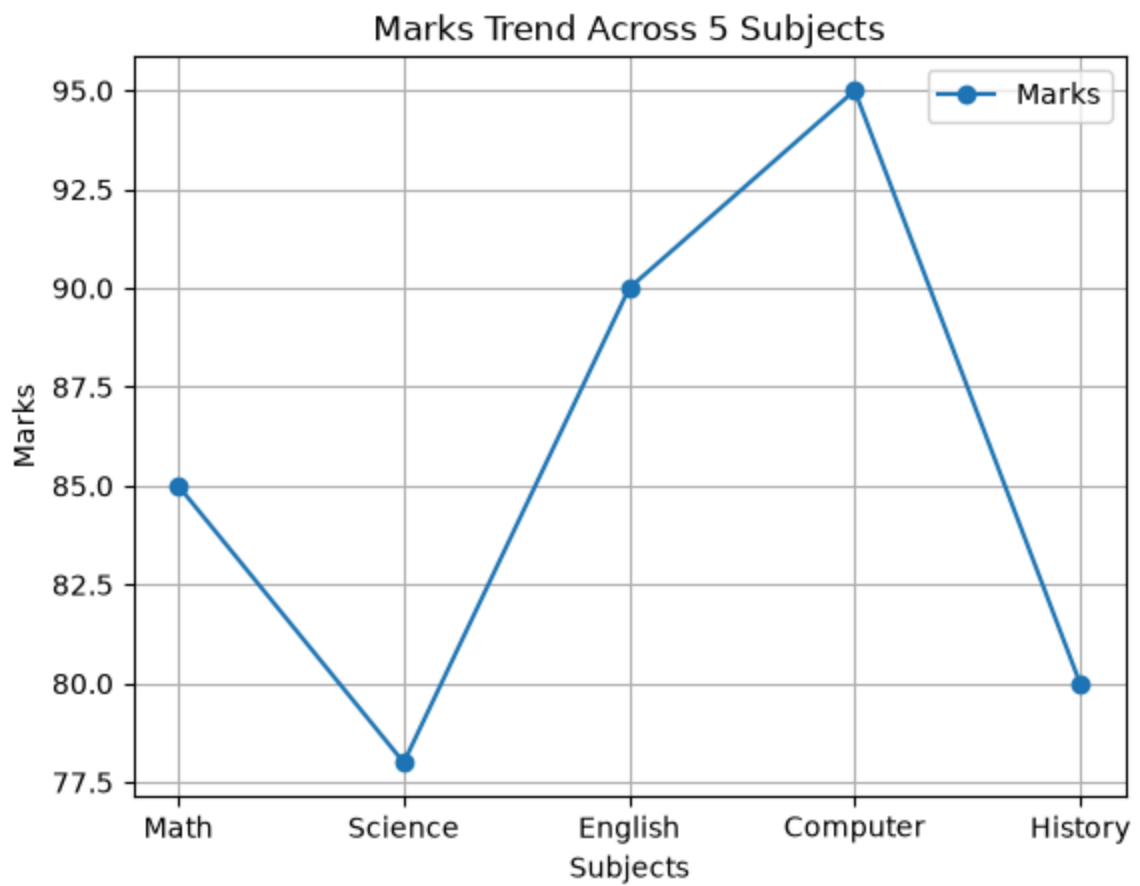
subjects = ["Math", "Science", "English", "Computer", "History"]
marks = [85, 78, 90, 95, 80]

plt.plot(subjects, marks, marker='o', label="Marks")
plt.title("Marks Trend Across 5 Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.legend()
plt.grid(True)
plt.show()

students = ["Aman", "Riya", "Sakshi", "Rahul", "Priya"]
student_marks = [85, 72, 91, 68, 88]

plt.bar(students, student_marks, label="Marks")
plt.title("Marks of 5 Students")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.legend()
plt.grid(True)
plt.show()

```

Q.10 Using Pandas built-in plotting, load or create a dataset with at least 4 numeric columns. Create:

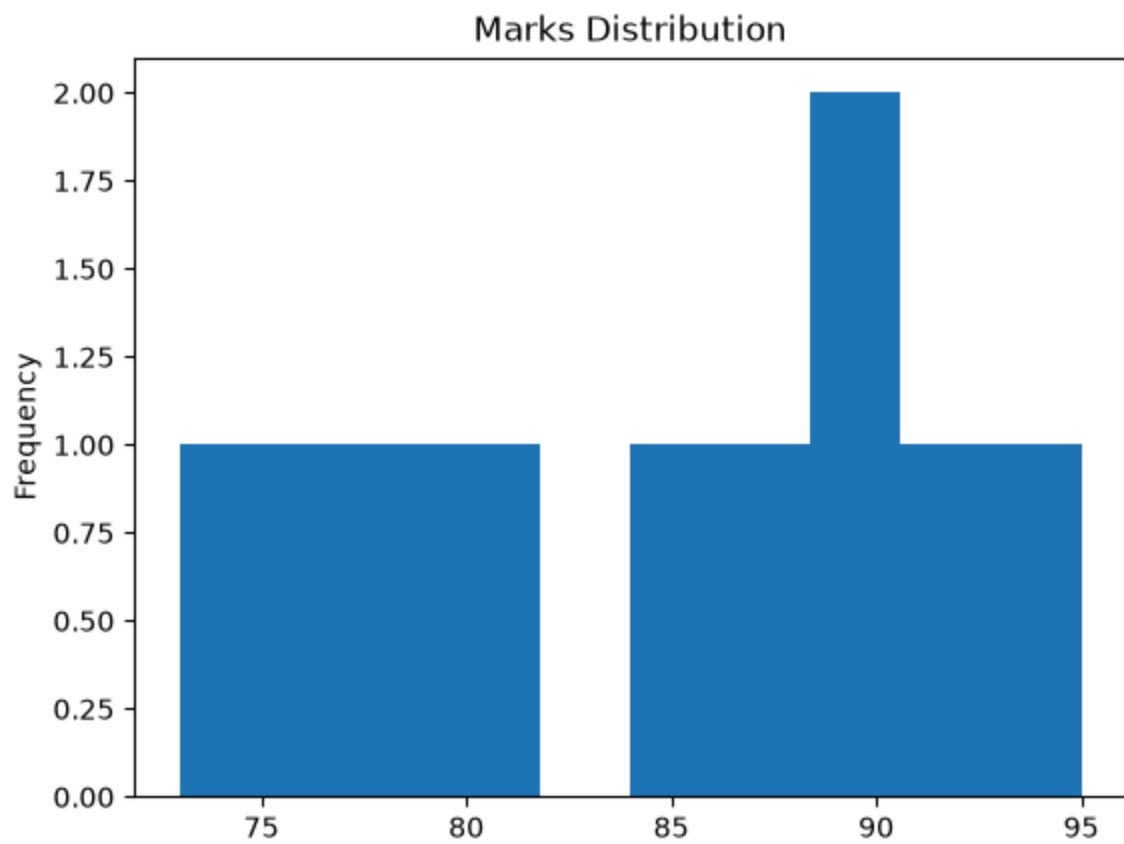
- A histogram for marks distribution
- A box plot to visualize spread and outliers
- A pie chart showing category-wise percentage

```
In [27]: import pandas as pd
data = {
    "Math": [85, 78, 92, 88, 76, 95, 81, 89, 73, 90],
    "Science": [80, 75, 90, 85, 70, 93, 79, 87, 72, 88],
    "English": [78, 82, 88, 84, 74, 91, 77, 86, 70, 89],
    "Computer": [90, 85, 95, 92, 80, 98, 88, 94, 79, 96],
    "Category": ["A", "B", "A", "B", "C", "A", "B", "A", "C", "A"]
}

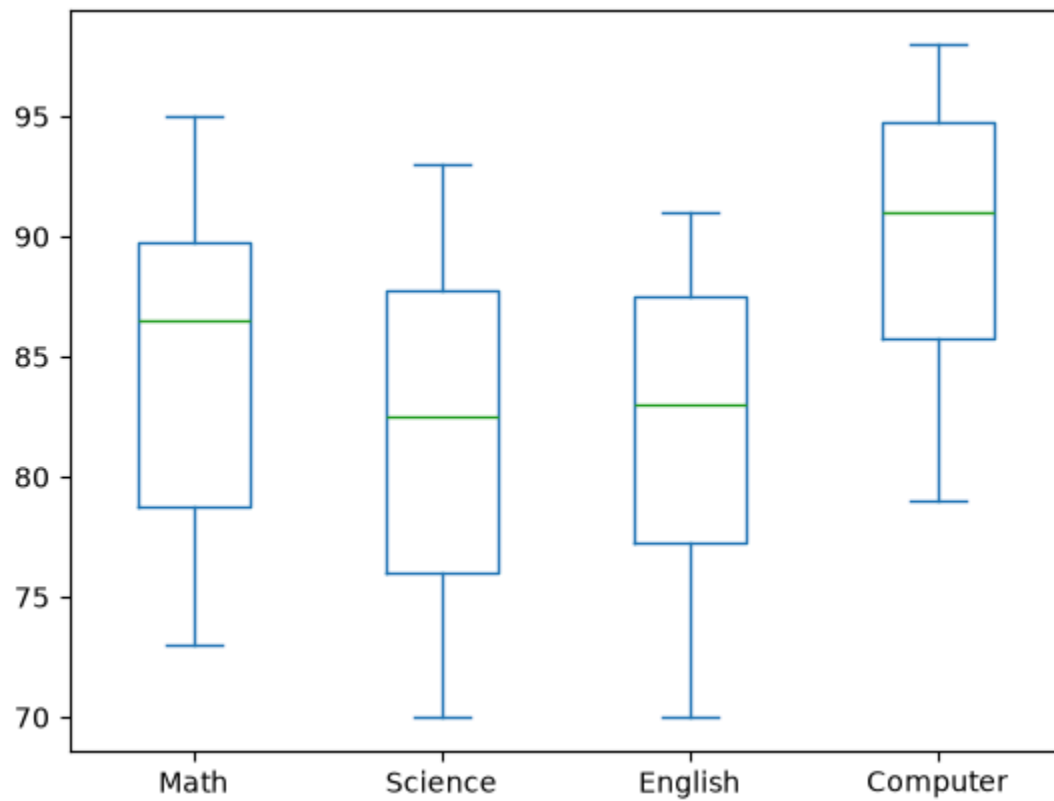
df = pd.DataFrame(data)
print(df)
```

	Math	Science	English	Computer	Category
0	85	80	78	90	A
1	78	75	82	85	B
2	92	90	88	95	A
3	88	85	84	92	B
4	76	70	74	80	C
5	95	93	91	98	A
6	81	79	77	88	B
7	89	87	86	94	A
8	73	72	70	79	C
9	90	88	89	96	A

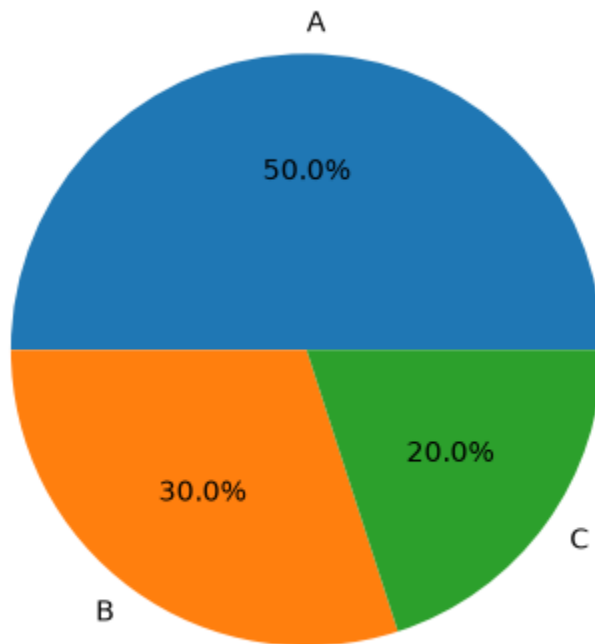
```
In [28]: df["Math"].plot(kind="hist", title="Marks Distribution")
plt.show()
```



```
In [29]: df[["Math", "Science", "English", "Computer"]].plot(kind="box")  
plt.show()
```



```
In [30]: df["Category"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.show()
```



Q.11 Using Seaborn, create a dataset of at least 50 records with columns Age, Salary, Department, Experience. Plot:

- A scatter plot of Age vs Salary with hue as Department
- A heatmap showing correlation between numeric columns
- A bar plot showing average Salary per Department

```
In [ ]: import seaborn as sns

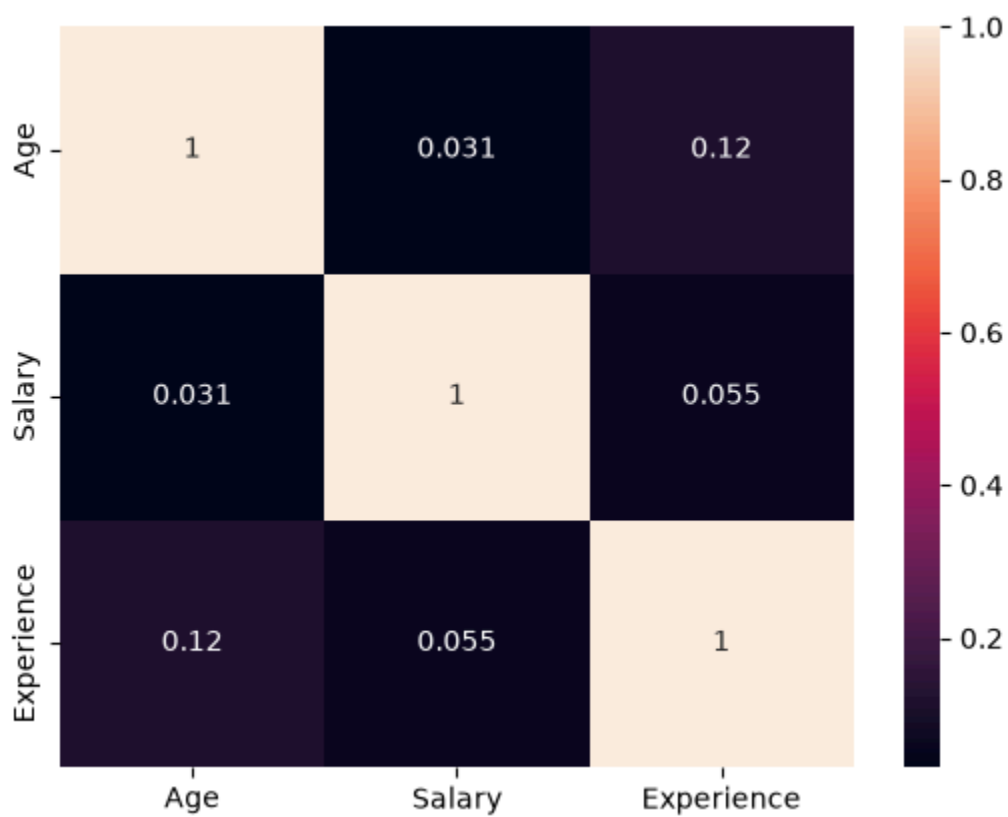
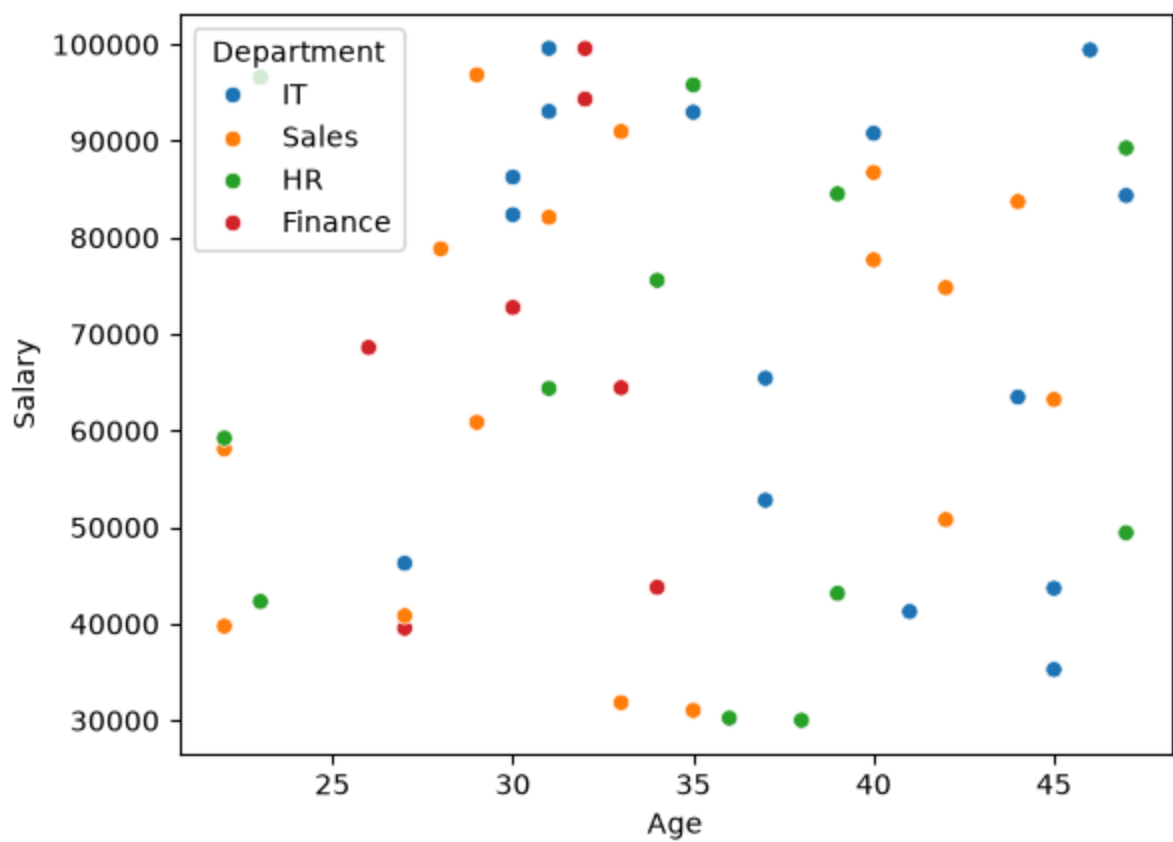
np.random.seed(1)

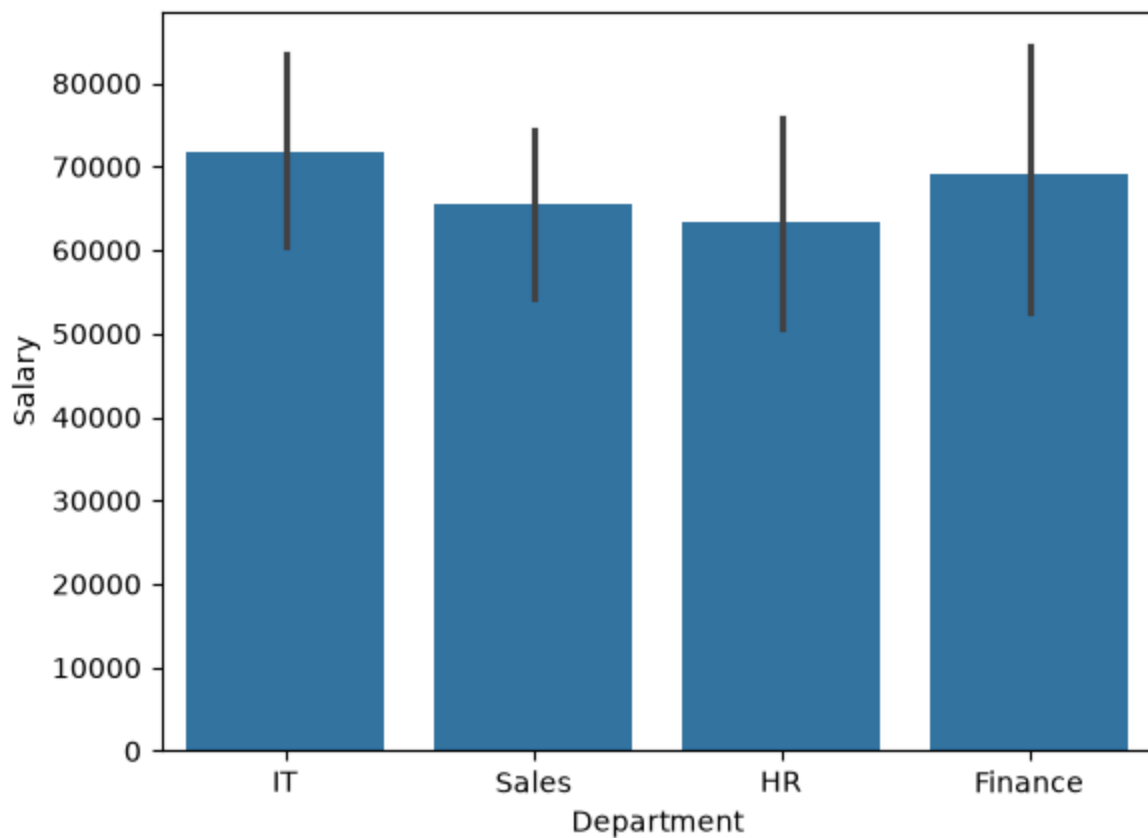
df = pd.DataFrame({
    "Age": np.random.randint(22, 50, 50),
    "Salary": np.random.randint(30000, 100000, 50),
    "Department": np.random.choice(["HR", "IT", "Sales", "Finance"], 50),
    "Experience": np.random.randint(1, 20, 50)
})

sns.scatterplot(x=df["Age"], y=df["Salary"], hue=df["Department"], color="green")
plt.show()

sns.heatmap(df[["Age", "Salary", "Experience"]].corr(), annot=True)
plt.show()

sns.barplot(data=df, x="Department", y="Salary")
plt.show()
```





Q.12 Create a subplot layout of 2 rows × 2 columns using Matplotlib containing:

- Line plot • Scatter plot • Bar chart • Histogram All using the same dataset. Add a main title using `suptitle()`

In [32]: `import matplotlib.pyplot as plt`

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]

fig, a = plt.subplots(2, 2)

a[0,0].plot(x, y)
a[0,0].set_title("Line")

a[0,1].scatter(x, y)
a[0,1].set_title("Scatter")

a[1,0].bar(x, y)
a[1,0].set_title("Bar")

a[1,1].hist(y)
a[1,1].set_title("Histogram")

plt.suptitle("All Plots")
plt.show()
```

All Plots

